FCyT
Sede Concepción del Uruguay

Licenciatura en Sistemas de Información
Arq. Programación Móvil - 2026

Flujo de login completo — iBank × Supabase

Implementar la interfaz completa de autenticación (inicio de sesión, registro y recuperación de contraseña) según el diseño de *iBank — Banking & E-Money Management App* en Figma, conectada a Supabase Auth como backend, respetando sus reglas de negocio y recomendaciones oficiales de seguridad.

DISEÑO	BACKEND	STACK SUGERIDO	ALCANCE
iBank UI Kit (Figma)	Supabase Auth (BaaS)	React Native + Expo	Solo UI + reglas de negocio

ÍNDICE

01 · Objetivo y alcance
02 · Recursos de diseño
03 · Pantallas a construir
04 · Stack técnico sugerido
05 · Configuración de Supabase
06 · Reglas de negocio por pantalla
07 · Reglas transversales
08 · Seguridad recomendada
09 · Definition of Done
10 · Rúbrica de evaluación
11 · Entregables
12 · Recursos oficiales

FCyT
Sede Concepción del Uruguay

Licenciatura en Sistemas de Información
Arq. Programación Móvil - 2026

01 · Objetivo y alcance

Construir el flujo de autenticación completo de una app bancaria, tal como está diseñado en el kit de Figma iBank, usando Supabase como único backend (no se desarrolla API propia). El foco del trabajo es la UI y sus reglas de negocio: cada pantalla debe verse fiel al diseño y comportarse exactamente como Supabase Auth espera que se comporte un cliente bien construido.

Quedan dentro del alcance obligatorio: inicio de sesión, registro y recuperación de contraseña, con todos sus estados (carga, error, éxito, deshabilitado) y validaciones. Quedan fuera de este TP —salvo que se sumen como puntos extra— el login social, la verificación por OTP como segundo factor y el PIN/biometría local, ya que no forman parte del alcance funcional definido para esta entrega.

02 · Recursos de diseño

El punto de partida es el frame indicado por `node-id=2-20347` dentro del archivo iBank — Banking & E-Money Management App. Este documento no reemplaza al Figma: las medidas exactas, tipografías, colores y estados de cada componente se extraen del archivo, no de esta consigna.

> CÓMO TRABAJAR EL ARCHIVO
>
> Abrí el archivo en modo Dev Mode (o inspeccioná con el panel de código) para copiar valores exactos: paleta de color en hex, familia y pesos tipográficos, radios de borde, espaciados y el spacing entre elementos de cada formulario. Relevá también los estados de inputs y botones que el kit incluya —vacío, foco, error, deshabilitado, cargando— porque son los que después hay que reproducir con las reglas de negocio de la sección 6.

Si el archivo incluye pantallas adicionales de autenticación (verificación OTP, login social, onboarding, PIN), tomalas como referencia de estilo para mantener consistencia visual, pero no son parte de los entregables obligatorios de este TP.

03 · Pantallas a construir

El flujo mínimo obligatorio consta de cinco pantallas. Usá los nombres de frame que tenga el Figma; los nombres acá son funcionales, no literales.

01 · Iniciar sesión Email + contraseña. Link a “Olvidé mi contraseña” y a Registro.	**02 · Registro** Alta de cuenta nueva con validación de contraseña en tiempo real.
03 · Confirmación pendiente Estado post-registro: “revisá tu email”, con opción de reenvío.	**04 · Recuperar contraseña** Solicitud de reset por email, con mensaje neutro anti-enumeración.
05 · Nueva contraseña Se abre desde el enlace del email. Define y confirma la contraseña nueva.	

04 · Stack técnico sugerido

- React Native + Expo (última SDK estable), fiel al carácter mobile del kit de diseño.
- `@supabase/supabase-js v2` como cliente oficial de Supabase.
- `@react-native-async-storage/async-storage` para persistir la sesión, tal como recomienda el quickstart oficial de Supabase para React Native. Como alternativa más segura para el refresh token podés evaluar `expo-secure-store`; documentá cuál elegiste y por qué.
- `expo-linking` para generar y capturar los deep links de confirmación de email y reset de contraseña.
- react-hook-form + zod (u otra combinación equivalente) para las validaciones de formulario del lado del cliente.
- `expo-router` (o React Navigation) para manejar la navegación y las rutas protegidas descritas en la sección 7.

05 · Configuración de Supabase

Cliente — lib/supabase.ts

```
// react-native-url-polyfill es requerido por supabase-js en RN
import 'react-native-url-polyfill/auto'
import AsyncStorage from '@react-native-async-storage/async-storage'
import { createClient } from '@supabase/supabase-js'

const supabaseUrl = process.env.EXPO_PUBLIC_SUPABASE_URL!
const supabaseAnonKey = process.env.EXPO_PUBLIC_SUPABASE_ANON_KEY!

export const supabase = createClient(supabaseUrl, supabaseAnonKey, {
  auth: {
    storage: AsyncStorage,
    autoRefreshToken: true,
    persistSession: true,
    detectSessionInUrl: false, // RN maneja el deep link a mano
  },
})
```

Ciclo de vida de la app — refresh en foreground/background

```
import { AppState } from 'react-native'
import { supabase } from './lib/supabase'

AppState.addEventListener('change', (state) => {
  if (state === 'active') supabase.auth.startAutoRefresh()
  else supabase.auth.stopAutoRefresh()
})
```

Scheme para deep links — app.json

```
{
  "expo": {
    "scheme": "ibanktp"
  }
}
```

> DASHBOARD DE SUPABASE
> Antes de tocar código, configurá el proyecto en Authentication: activá “Confirm email”, definí la política de contraseñas, revisá los rate limits por defecto y agregá el scheme (ibanktp://reset-password, ibanktp://confirm) a la lista de Redirect URLs permitidas en URL Configuration. El detalle completo está en la sección 8.

06 · Reglas de negocio por pantalla

Esta es la parte central del TP: cada pantalla no es solo un layout, es un conjunto de reglas que hay que reproducir con precisión porque son las que Supabase realmente aplica del lado del servidor.

6.1 Iniciar sesión — signInWithPassword

- Email con formato válido y password no vacía habilitan el botón; mientras no se cumplan, el botón queda deshabilitado (no oculto).
- Durante la request: inputs deshabilitados, botón con spinner y texto de carga.
- `Invalid login credentials` → mostrar un único mensaje genérico (“Email o contraseña incorrectos”), sin indicar cuál de los dos falló. Es la misma lógica anti-enumeración que aplica Supabase del lado del servidor.
- `Email not confirmed` → redirigir a la pantalla de Confirmación pendiente (6.3) en lugar de mostrar un error suelto.
- Error `429` (rate limit) → mensaje de “demasiados intentos” y cooldown visual de 60 segundos en el botón, alineado al límite por defecto de Supabase para reintentos del mismo usuario.
- Login exitoso → la sesión ya queda persistida por la configuración del cliente (sección 5); navegar a Home. Si al abrir la app ya existe una sesión válida, esta pantalla ni se muestra (ver 7.2).

6.2 Registro — signUp

- Validar en el cliente las mismas reglas de contraseña configuradas en el dashboard (ver 8): longitud mínima, mayúscula, minúscula, dígito y símbolo. Mostrarlas como checklist visual que se va tildando mientras el usuario escribe, no como un único mensaje de error al enviar.

- Campo “confirmar contraseña” debe coincidir exactamente (validación de cliente, Supabase no la conoce).
- Checkbox de aceptación de Términos obligatorio para habilitar el botón; Supabase no valida esto, es responsabilidad exclusiva de la UI.
- Llamada con `options.data` para guardar nombre u otros campos del formulario en `user_metadata`, y `options.emailRedirectTo` apuntando al deep link de confirmación.
- Con “Confirm email” activo, reintentar el registro de un email ya existente no debe revelar que la cuenta existe: Supabase reenvía la confirmación o responde sin error explícito según el caso. La UI debe mostrar siempre el mismo mensaje neutro de éxito (“Revisá tu email para continuar”).
- Registro exitoso → navegar a Confirmación pendiente (6.3).

6.3 Confirmación pendiente — resend

- Mostrar el email al que se envió el enlace (no editable desde acá).
- Botón “Reenviar email” que llama a `supabase.auth.resend({ type: 'signup', email })`, con el mismo cooldown de 60 segundos que el resto de los envíos de email.
- Al volver del deep link de confirmación con una sesión válida, redirigir automáticamente a Home sin pasos intermedios.

6.4 Recuperar contraseña — resetPasswordForEmail

```
import * as Linking from 'expo-linking'

const redirectTo = Linking.createURL('reset-password')

await supabase.auth.resetPasswordForEmail(email, { redirectTo })
```

- Único campo: email. Validar solo formato, nada más.
- Regla de negocio no negociable: la UI debe mostrar exactamente el mismo mensaje de éxito exista o no una cuenta con ese email (“Si el email existe en nuestro sistema, vas a recibir instrucciones”). Supabase ya se comporta así del lado del servidor para prevenir enumeración de usuarios; romper esto en la UI —por ejemplo mostrando “email no encontrado”— anula esa protección.
- Mismo cooldown de 60 segundos entre reintentos para el mismo email.

6.5 Nueva contraseña — onAuthStateChange + updateUser

```
supabase.auth.onAuthStateChange((event, session) => {
  if (event === 'PASSWORD_RECOVERY') {
    // hay una sesión temporal de recuperación: navegar al formulario
  }
})

// al enviar el formulario
const { error } = await supabase.auth.updateUser({ password: nuevaPassword })
```

- Esta pantalla solo se llega a través del deep link del email; capturá la URL entrante con expo-linking y esperá el evento PASSWORD_RECOVERY antes de mostrar el formulario.
- Si el link llegó vencido o inválido, mostrar un estado de error propio con un link para volver a pedir el reset (6.4), nunca el formulario de contraseña vacío por defecto.
- Mismas reglas de complejidad de contraseña que en Registro, más confirmación de contraseña.
- Tras un `updateUser` exitoso, cerrar la sesión de recuperación (`signOut`) y llevar al usuario a Login con un mensaje de éxito, en lugar de dejarlo logueado automáticamente con esa sesión temporal.

07 · Reglas transversales

- 7.1 Estados de carga uniformes: todo botón de submit tiene estado loading (spinner + disabled) y todo input se deshabilita durante la request. Ningún formulario permite doble envío.
- 7.2 Rutas protegidas: mientras se resuelve la sesión al abrir la app, mostrar un estado de carga (nunca un parpadeo del login). Con sesión válida, las pantallas de autenticación quedan inaccesibles y redirigen a Home; sin sesión, ocurre lo inverso.
- 7.3 Logout: `supabase.auth.signOut()` limpia el storage local y vuelve a Login.
- 7.4 Mapeo de errores centralizado: un único punto (hook o util) traduce los códigos de error de Supabase (invalid_credentials, email_not_confirmed, over_request_rate_limit, weak_password, user_already_exists, errores de red) a los mensajes en español que se muestran en cada pantalla, para no duplicar esa lógica en cada formulario.

- 7.5 Nada sensible en logs: ni contraseñas ni tokens se imprimen en consola, ni quedan en el estado de la app más tiempo del necesario.

08 · Seguridad recomendada (Dashboard de Supabase)

Configuración a revisar en Authentication antes de dar el TP por terminado, siguiendo la documentación oficial de Supabase:

ÍTEM	RECOMENDACIÓN
Longitud mínima de contraseña	8 caracteres como piso; Supabase sugiere no bajar de ahí.
Complejidad de contraseña	Exigir mayúscula, minúscula, dígito y símbolo.
Leaked password protection	Activarla (usa la API de HaveIBeenPwned) — requiere plan Pro o superior.
Confirm email	Activo en producción; puede desactivarse solo durante desarrollo local.
Rate limits	60s de cooldown por usuario en signup/recover por defecto; ajustar solo si el caso lo justifica.
Expiración de link/OTP	3600 segundos (1 hora) o menos.
SMTP	Custom SMTP para producción — el proveedor incluido limita a 2 emails/hora.
Redirect URLs	Agregar el scheme de la app a la allowlist de URL Configuration.
CAPTCHA	Opcional/bonus: hCaptcha o Turnstile en signup, signin y reset. En RN se integra vía WebView, pasando el token en options.captchaToken.
Claves	Solo la anon/publishable key vive en el cliente. La service_role nunca se incluye en la app.

> **FUERA DE ALCANCE, PERO RELEVANTE**
>
> Si el registro pide datos además de email y contraseña (nombre, foto), la práctica recomendada por Supabase es guardarlos en `user_metadata` al hacer `signUp` y, si hace falta consultarlos desde otras tablas, sincronizarlos a una tabla `profiles` pública mediante un trigger, protegida con Row Level Security (`auth.uid() = id`). No es obligatorio para este TP —foco 100% UI— pero suma como punto extra si se documenta.

09 · Definition of Done

- ☐ Las 5 pantallas están implementadas y son fieles al Figma (colores, tipografía, espaciados extraídos del archivo real).
- ☐ Todas las validaciones de cliente descritas en la sección 6 están implementadas y coinciden con la configuración del dashboard.
- ☐ Los mensajes de error de la sección 6 y 7.4 están cubiertos, incluyendo el comportamiento anti-enumeración en login, registro y reset.
- ☐ Los deep links de confirmación de email y de reset de contraseña funcionan de punta a punta.
- ☐ Las rutas protegidas evitan que un usuario logueado vea pantallas de auth, y viceversa.
- ☐ La sesión persiste después de cerrar y reabrir la app.
- ☐ No hay contraseñas, tokens ni la `service_role key` expuestos en el código o en logs.
- ☐ El flujo fue probado en al menos una plataforma (iOS o Android) y quedó documentado si no se probó en la otra.

10 · Rúbrica de evaluación

CRITERIO	DESCRIPCIÓN	PTS
Fidelidad visual	Coincidencia con el Figma: color, tipografía, spacing, estados de componentes.	20
Integración con Supabase	Uso correcto de la API de Auth y manejo de sesión (persistencia, refresh, logout).	20
Cobertura del flujo	Las 5 pantallas y la navegación entre ellas funcionan de punta a punta.	15
Validaciones de formulario	Reglas de contraseña, formato de email, confirmación de contraseña, checkbox de términos.	15
Reglas de negocio y errores	Anti-enumeración, rate limiting en UI, mapeo de errores, estados de confirmación pendiente.	15
Seguridad y configuración	Dashboard configurado según la sección 8, sin secretos expuestos.	10
Código y accesibilidad	Componentes reutilizables, estados de foco visibles, tamaños de touch target adecuados.	5

11 · Entregables

- Repositorio con el código fuente y un README con instrucciones de instalación, variables de entorno necesarias y cómo correr el proyecto.
- Capturas de pantalla o una grabación corta mostrando las 5 pantallas y los flujos de error/éxito más relevantes.
- Documento breve de decisiones: qué se mapeó del Figma tal cual, qué se adaptó y por qué, y qué quedó fuera de alcance.
- Configuración de Supabase documentada: valores usados en la política de contraseñas, rate limits si se modificaron, y redirect URLs configuradas.

12 · Recursos oficiales

- Use Supabase Auth with React Native
- Password-based Auth
- Password security
- resetPasswordForEmail — JS reference
- Rate limits
- Production checklist
- Enable CAPTCHA protection
- Using Supabase — Expo docs